

CSCI 135 Assignment – Hello Arguments (and variables)

CSCI 135 Assignment – Hello Arguments (and variables)

August 30th / September 1st, 2023

Due Date: Sunday, 9/3/2023 11:59PM

Points: 20

Topic: Introduction to Arguments and Variables in Python

The goal of this assignment is to introduce you to arguments and variables in Python. You will learn how to pass command line arguments to your Python scripts.

Submission:

After following the instructions below, please submit the two Python scripts in Moodle. Upload these as separate files – do not zip them. Be sure each file has the recommended header including the file name, your name, and a description of the program.

- HelloArgs.py
- Stairway.py

Grading:

Each program/script is worth 10 points each for a total of 20 points.

For each program, you will be graded according to the following criteria:

	HelloArgs.py	Stairway.py
Program Runs	2	2
Header Comment	2	2
Program Runs Correctly	4	4
Correctly Formatted Output	2	2
Total	10	10

Part 1: Hello Arguments!

Create a script named `HelloArgs.py` that takes four command line arguments. The first command line argument is a general greeting word such as "Hello" or "Greetings". The second, third, and fourth command line arguments are the names of three people. The program outputs three lines of text. Each line of text has the greeting word, followed by a space, followed in order by the name of one of the three people, followed by an exclamation mark.

Here is an example run of the script:

```
C:\Users\dgalarus> python HelloArgs.py Hello Keith Justin Joy
```

Output:

```
Hello Keith!
Hello Justin!
Hello Joy!
```

A key difference of this program versus `HelloWorld.py` and `Greetings.py` from the first lab is that the output of `HelloArgs.py` depends on the input from the command line. Running the exact same program but with the different command line arguments "Hola Alice Bob Carol" results in the output:

```
Hola Alice!
Hola Bob!
Hola Carol!
```

What is a command line argument? You need to get input into your script in some way, and this is a way of doing that. In class we saw the `archive.py` script which asks the user to enter a web address and date interactively. Command line arguments are another way of providing input and are useful in situations where a script may accept input but direct interaction from a user is not needed.

How do you access the command line arguments inside your Python scripts? See `LikeDontLike.py`, and `LikeDontLike2.py` for basic examples of using command line input inside the program. (These are provided for you in Moodle in the folder named `LabExamples`. You may download them from Moodle.) Open these scripts for viewing in Notepad++.

There are several things to notice in `LikeDontLike.py`, and `LikeDontLike2.py`:

- The `import sys` line imports the `sys` package, which gives the script access to the command line arguments.
- The command line arguments are referenced directly as `sys.argv[1]`, `sys.argv[2]`, etc.

The `LikeDontLike2.py` script references the arguments with variables and uses the variables in the subsequent print statement.

CSCI 135 Assignment – Hello Arguments (and variables)

Run these scripts to see how they work. Example usage:

```
> python LikeDontLike.py steak sushi
```

```
> python LikeDontLike2.py dogs cats
```

You need to be careful so that your script creates the specified output. This is important in your assignments and in general when writing software for others. While programming and designing software involves creative elements, you need to do what you and your customer / boss / instructor agree to.

Here are some other things to watch out for:

- What if your program's output does not look the same as specified in the examples? Is this okay? No. Programming requires attention to detail. Strive to match specified and example outputs exactly. There will be opportunities for creativity later in the semester.
- What if you didn't name your program exactly as specified? If you name your program different than what was specified, you will lose points. There may be times when the TAs or I use automated methods (scripts) to process your homework. These methods will look for the exact file names that we expect and will fail if they can't find them. This even includes matching case. For our purposes, the specified script name `HelloArgs.py` is not the same as `helloargs.py`. While Microsoft Windows is case-insensitive when it comes to file names, other operating systems such as Linux and Mac are case-sensitive. Matching case is an important part of correctly naming a file so it will correctly run on all systems.
- What if you run your program, it crashes, and you get an error message that says `IndexError: list index out of range`? What does this mean? It likely means that you did not provide enough arguments at the command line. You do NOT need to include error handling in this assignment. You can assume the user will provide exactly four command line arguments. (Note that your script likely will not crash if you provide more than four arguments.)

Upload HelloArgs.py to Moodle as the first part of your submission for this assignment.

Part 2: Stairway

Create a program `Stairway.py` that calculates the total number of steps in a building's stairway and the total vertical height of the stairway. The program takes three command line arguments as input in the following order:

- Number of flights of stairs in the building: an integer (example, 10).
- Number of steps per flight: an integer (example, 13).
- Vertical height of a step, in feet: a floating-point number (example, 0.75).

Here is an example run of the script:

```
> python Stairway.py 10 13 0.75
```

Output:

```
Total steps: 130
```

```
Total height in feet: 97.5
```

Another example:

```
> python Stairway.py 5 21 0.81
```

Output:

```
Total steps: 105
```

```
Total height in feet: 85.050000000000001
```

How do you read integer or floating-point command line arguments? The command line input, `sys.argv[1]`, ..., `sys.argv[3]`, is read as text. You cannot directly perform mathematical calculations using text, so you must convert the first two command line arguments to `int` variables, and the third argument into a `float` variable. For an example of how this should be done, see `ArgsExample.py`. Do not use the `eval()` function that Liang uses in his examples.

Why is a strange value output for the total height in the second example above? Computers represent data using binary representations that occupy a finite amount of space. Sometimes fractions and decimal numbers don't convert exactly to fit that representation. It is good to be aware of this. We will eventually learn how to format floating-point numbers nicely, but for now we'll leave it as is.

Why not represent the number of flights or the number of steps per flight as a float instead of an int? The program still could work, but it is good practice to use a data type that best represents the data. Our program isn't intended to support partial flights of stairs, or flights that contain partial steps. If we used float variables, someone using our program might think that such things were supported.

Upload Stairway.py to Moodle as the second part of your submission for this assignment.

EXTENSIONS

I will try to add “extensions” to your assignments to give you further opportunity to explore, learn and practice. You will not submit answers to these extensions as part of your assignment submission. You should work through and understand them to learn more and practice. View these an opportunity, not a burden.

From the Liang book - NOTE: **We will not use MyProgrammingLab in the book.** You should be able to do exercises from Liang without MyProgrammingLab.:

1. *Answer the checkpoint and other inline questions in Chapter 2.*
2. *Read about reading and using input from the console. Note how this is different from using command line arguments. We will use console input in subsequent labs, but you can begin to understand and practice with it now. NOTE: Liang’s use of the eval() function can be problematic for a variety of reasons. I will discuss this later in the semester. Be careful in your use of eval() and watch for better / alternative ways of dealing with type conversions and parsing. When asked to not use eval() (see above), use a better option.*
3. *Try the code examples Chapter 2. Understand what they do and how they do it.*
4. *Try exercises 2.1 – 2.5 as specified by Liang using console input. When done, try these exercises again, but used command line arguments for the input(s) instead.*
5. *Try some of the other programming exercises at the end of Chapter 2.*

From the Things to Do section at the end of Chapter Two of our optional Lubanovic book:

2.1 *Assign the integer value 99 to the variable prince, and print it.*

2.2 *What type is the value 5?*

2.3 *What type is the value 2.0?*

2.4 *What type is the expression 5 + 2.0?*

From me:

1. If you run the following command with the example file you will not get the desired output:

```
> python LikeDontLike.py Scottish Terriers Dalmations
```

Desired output:

```
I like Scottish Terriers, but I don't like Dalmations.
```

Without changing the LikeDontLike.py script, how can you get it to produce the desired output?

2. For any given Python script, what is the value of `sys.argv[0]`?

How might this be useful?